

第8章 使用寄存器点亮 LED 灯

本章参考资料：《STM32F4xx 中文参考手册》，学习本章时，配合《STM32F4xx 中文参考手册》“通用 I/O(GPIO)”章节一起阅读，效果会更佳，特别是涉及到寄存器说明的部分。关于建立工程时使用 KEIL5 的基本操作，请参考前面的章节。

8.1 GPIO 简介

GPIO 是通用输入输出端口的简称，简单来说就是 STM32 可控制的引脚，STM32 芯片的 GPIO 引脚与外部设备连接起来，从而实现与外部通讯、控制以及数据采集的功能。STM32 芯片的 GPIO 被分成很多组，每组有 16 个引脚，如型号为 STM32F41GT6 型号的芯片有 GPIOA、GPIOB、GPIOC 至 GPIOG 共 7 组 GPIO，芯片一共 144 个引脚，其中 GPIO 就占了一大部分，所有的 GPIO 引脚都有基本的输入输出功能。

最基本的输出功能是由 STM32 控制引脚输出高、低电平，实现开关控制，如把 GPIO 引脚接入到 LED 灯，那就可以控制 LED 灯的亮灭，引脚接入到继电器或三极管，那就可以通过继电器或三极管控制外部大功率电路的通断。

最基本的输入功能是检测外部输入电平，如把 GPIO 引脚连接到按键，通过电平高低区分按键是否被按下。

8.2 GPIO 框图剖析

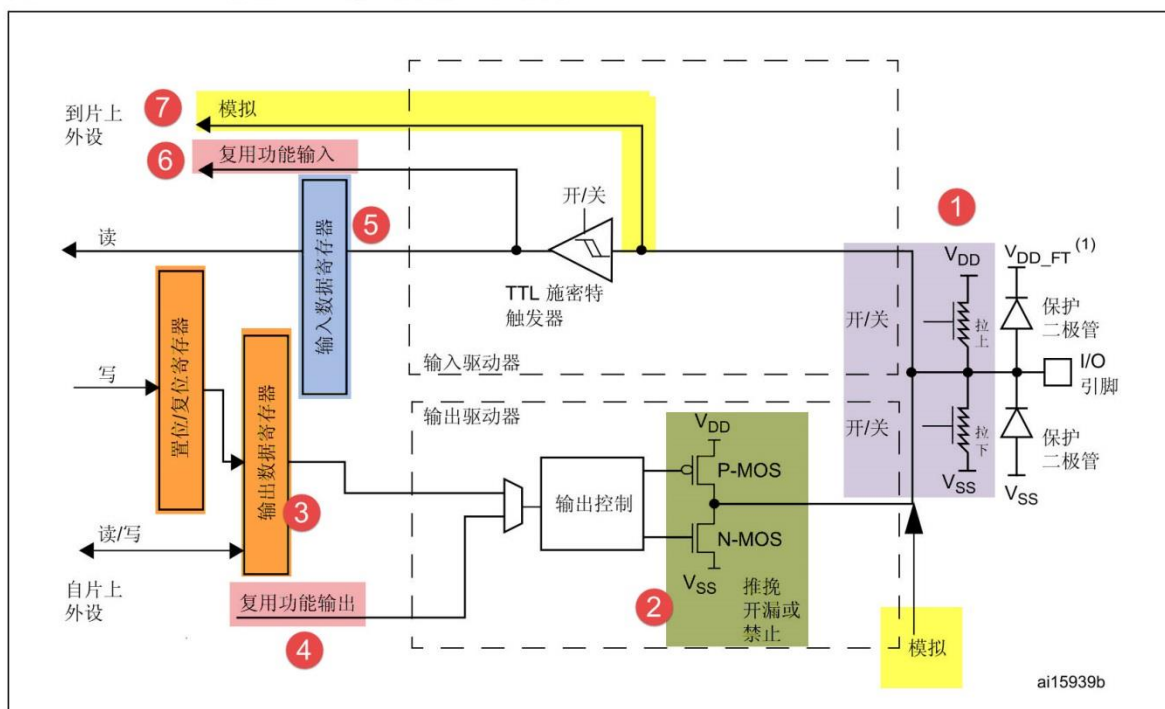


图 8-1 GPIO 结构框图

通过 GPIO 硬件结构框图，就可以从整体上深入了解 GPIO 外设及它的各种应用模式。该图从最右端看起，最右端就是代表 STM32 芯片引出的 GPIO 引脚，其余部件都位于芯片内部。

8.2.1 基本结构分析

下面我们按图中的编号对 GPIO 端口的结构部件进行说明。

1. 保护二极管及上、下拉电阻

引脚的两保护个二极管可以防止引脚外部过高或过低的电压输入，当引脚电压高于 V_{DD_FT} 时，上方的二极管导通，当引脚电压低于 V_{SS} 时，下方的二极管导通，防止不正常电压引入芯片导致芯片烧毁。尽管有这样的保护，并不意味着 STM32 的引脚能直接外接大功率驱动器件，如直接驱动电机，强制驱动要么电机不转，要么导致芯片烧坏，必须要加大功率及隔离电路驱动。具体电压、电流范围可查阅《STM32F4xx 英文数据手册》。

上拉、下拉电阻，从它的结构我们可以看出，通过上、下拉对应的开关配置，我们可以控制引脚默认状态的电压，开启上拉的时候引脚电压为高电平，开启下拉的时候引脚电压为低电平，这样可以消除引脚不定状态的影响。如引脚外部没有外接器件，或者外部的器件不干扰该引脚电压时，STM32 的引脚都会有这个默认状态。

也可以设置“既不上拉也不下拉模式”，我们也把这种状态称为浮空模式，配置成这个模式时，直接用电压表测量其引脚电压为 1 点几伏，这是个不确定值。所以一般来说我们都会选择给引脚设置“上拉模式”或“下拉模式”使它有默认状态。

STM32 的内部上拉是“弱上拉”，即通过此上拉输出的电流是很弱的，如要求大电流还是需要外部上拉。

通过“上拉/下拉寄存器 $GPIOx_PUPDR$ ”控制引脚的上、下拉以及浮空模式。

2. P-MOS 管和 N-MOS 管

GPIO 引脚线路经过两个保护二极管后，向上流向“输入模式”结构，向下流向“输出模式”结构。先看输出模式部分，线路经过一个由 P-MOS 和 N-MOS 管组成的单元电路。这个结构使 GPIO 具有了“推挽输出”和“开漏输出”两种模式。

所谓的推挽输出模式，是根据这两个 MOS 管的工作方式来命名的。在该结构中输入高电平时，经过反向后，上方的 P-MOS 导通，下方的 N-MOS 关闭，对外输出高电平；而在该结构中输入低电平时，经过反向后，N-MOS 管导通，P-MOS 关闭，对外输出低电平。当引脚高低电平切换时，两个管子轮流导通，P 管负责灌电流，N 管负责拉电流，使其负载能力和开关速度都比普通的方式有很大的提高。推挽输出的低电平为 0 伏，高电平为 3.3 伏，具体参考图 8-2，它是推挽输出模式时的等效电路。

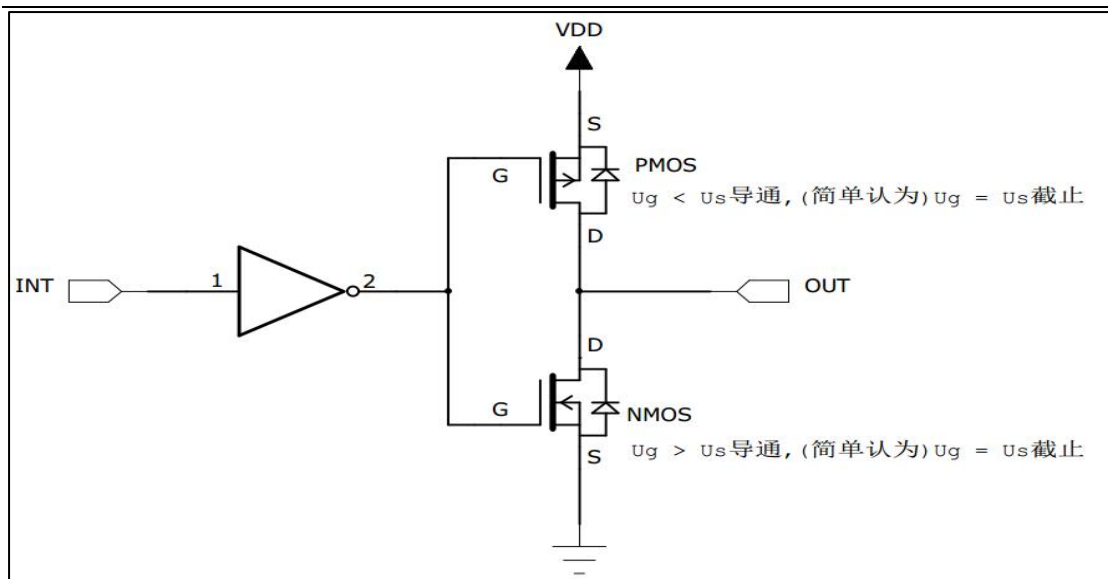


图 8-2 推挽等效电路

而在开漏输出模式时，上方的 P-MOS 管完全不工作。如果我们控制输出为 0，低电平，则 P-MOS 管关闭，N-MOS 管导通，使输出接地，若控制输出为 1（它无法直接输出高电平）时，则 P-MOS 管和 N-MOS 管都关闭，所以引脚既不输出高电平，也不输出低电平，为高阻态。为正常使用时必须外部接上拉电阻，参考图 8-3 中等效电路。它具有“线与”特性，也就是说，若有很多个开漏模式引脚连接到一起时，只有当所有引脚都输出高阻态，才由上拉电阻提供高电平，此高电平的电压为外部上拉电阻所接的电源的电压。若其中一个引脚为低电平，那线路就相当于短路接地，使得整条线路都为低电平，0 伏。

推挽输出模式一般应用在输出电平为 0 和 3.3 伏而且需要高速切换开关状态的场合。在 STM32 的应用中，除了必须用开漏模式的场合，我们都习惯使用推挽输出模式。

开漏输出一般应用在 I2C、SMBUS 通讯等需要“线与”功能的总线电路中。除此之外，还用在不匹配电平的情况，如需要输出 5 伏的高电平，就可以在外部接一个上拉电阻，上拉电源为 5 伏，并且把 GPIO 设置为开漏模式，当输出高阻态时，由上拉电阻和电源向外输出 5 伏的电平，具体见图 8-4。

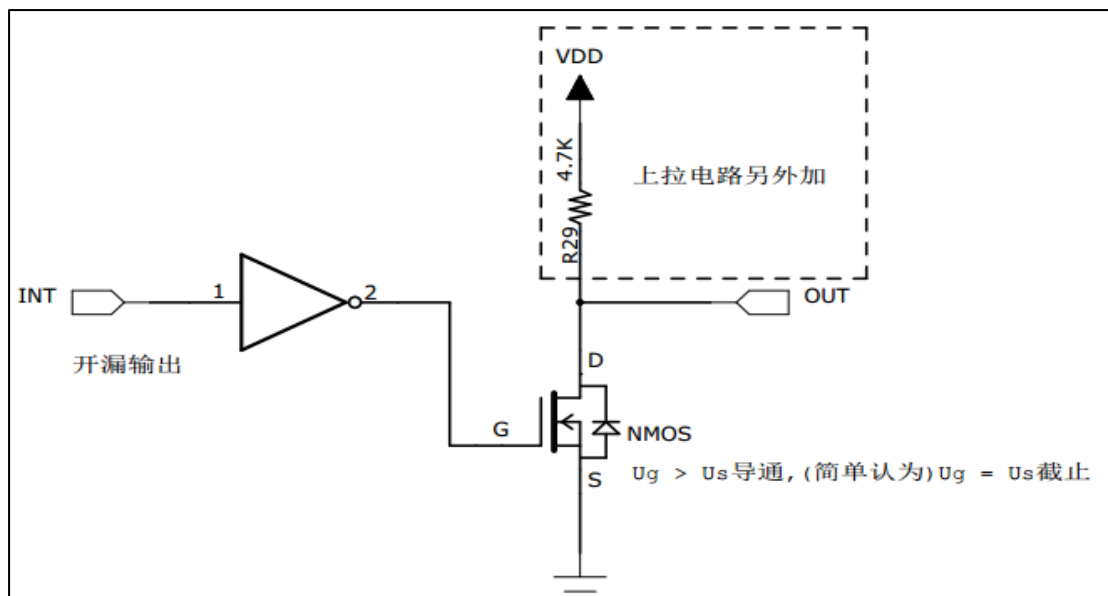


图 8-3 开漏电路

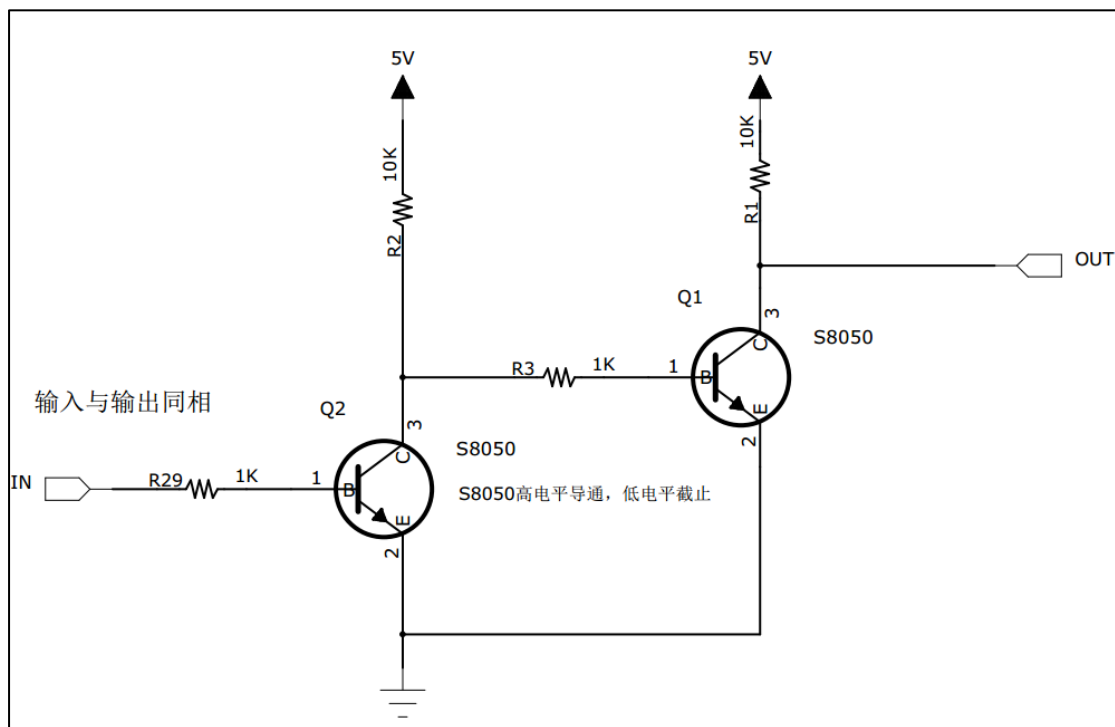


图 8-4 STM32 IO 对外输出 5V 电平

3. 输出数据寄存器

前面提到的双 MOS 管结构电路的输入信号，是由 GPIO “输出数据寄存器 GPIOx_ODR” 提供的，因此我们通过修改输出数据寄存器的值就可以修改 GPIO 引脚的输出电平。而“置位/复位寄存器 GPIOx_BSRR”可以通过修改输出数据寄存器的值从而影响电路的输出。

4. 复用功能输出

“复用功能输出”中的“复用”是指 STM32 的其它片上外设对 GPIO 引脚进行控制，此时 GPIO 引脚用作该外设功能的一部分，算是第二用途。从其它外设引出来的“复用功能输出信号”与 GPIO 本身的数据寄存器都连接到双 MOS 管结构的输入中，通过图中的梯形结构作为开关切换选择。

例如我们使用 USART 串口通讯时，需要用到某个 GPIO 引脚作为通讯发送引脚，这个时候就可以把该 GPIO 引脚配置成 USART 串口复用功能，由串口外设控制该引脚，发送数据。

5. 输入数据寄存器

看 GPIO 结构框图的上半部分，它是 GPIO 引脚经过上、下拉电阻后引入的，它连接到施密特触发器，信号经过触发器后，模拟信号转化为 0、1 的数字信号，然后存储在“输入数据寄存器 GPIOx_IDR”中，通过读取该寄存器就可以了解 GPIO 引脚的电平状态。

6. 复用功能输入

与“复用功能输出”模式类似，在“复用功能输入模式”时，GPIO 引脚的信号传输到 STM32 其它片上外设，由该外设读取引脚状态。

同样，如我们使用 USART 串口通讯时，需要用到某个 GPIO 引脚作为通讯接收引脚，这个时候就可以把该 GPIO 引脚配置成 USART 串口复用功能，使 USART 可以通过该通讯引脚的接收远端数据。

7. 模拟输入输出

当 GPIO 引脚用于 ADC 采集电压的输入通道时，用作“模拟输入”功能，此时信号是不经过施密特触发器的，因为经过施密特触发器后信号只有 0、1 两种状态，所以 ADC 外设要采集到原始的模拟信号，信号源输入必须在施密特触发器之前。类似地，当 GPIO 引脚用于 DAC 作为模拟电压输出通道时，此时作为“模拟输出”功能，DAC 的模拟信号输出就不经过双 MOS 管结构了，在 GPIO 结构框图的右下角处，模拟信号直接输出到引脚。同时，当 GPIO 用于模拟功能时(包括输入输出)，引脚的上、下拉电阻是不起作用的，这个时候即使在寄存器配置了上拉或下拉模式，也不会影响到模拟信号的输入输出。

8.2.2 GPIO 工作模式

总结一下，由 GPIO 的结构决定了 GPIO 可以配置成以下模式：

1. 输入模式(上拉/下拉/浮空)

在输入模式时，施密特触发器打开，输出被禁止。数据寄存器每隔 1 个 AHB1 时钟周期更新一次，可通过输入数据寄存器 GPIOx_IDR 读取 I/O 状态。其中 AHB1 的时钟如按默认配置一般为 180MHz。

用于输入模式时，可设置为上拉、下拉或浮空模式。

2. 输出模式(推挽/开漏，上拉/下拉)

在输出模式中，输出使能，推挽模式时双 MOS 管以方式工作，输出数据寄存器 GPIOx_ODR 可控制 I/O 输出高低电平。开漏模式时，只有 N-MOS 管工作，输出数据寄存器可控制 I/O 输出高阻态或低电平。输出速度可配置，有 2MHz\25MHz\50MHz\100MHz 的选项。此处的输出速度即 I/O 支持的高低电平状态最高切换频率，支持的频率越高，功耗越大，如果功耗要求不严格，把速度设置成最大即可。

此时施密特触发器是打开的，即输入可用，通过输入数据寄存器 GPIOx_IDR 可读取 I/O 的实际状态。

用于输出模式时，可使用上拉、下拉模式或浮空模式。但此时由于输出模式时引脚电平会受到 ODR 寄存器影响，而 ODR 寄存器对应引脚的位为 0，即引脚初始化后默认输出低电平，所以在这种情况下，上拉只起到小幅提高输出电流能力，但不会影响引脚的默认状态。

3. 复用功能(推挽/开漏，上拉/下拉)

复用功能模式中，输出使能，输出速度可配置，可工作在开漏及推挽模式，但是输出信号源于其它外设，输出数据寄存器 GPIOx_ODR 无效；输入可用，通过输入数据寄存器可获取 I/O 实际状态，但一般直接用外设的寄存器来获取该数据信号。

用于复用功能时，可使用上拉、下拉模式或浮空模式。同输出模式，在这种情况下，初始化后引脚默认输出低电平，上拉只起到小幅提高输出电流能力，但不会影响引脚的默认状态。

4. 模拟输入输出

模拟输入输出模式中，双 MOS 管结构被关闭，施密特触发器停用，上/下拉也被禁止。其它外设通过模拟通道进行输入输出。

通过对 GPIO 寄存器写入不同的参数，就可以改变 GPIO 的应用模式，再强调一下，要了解具体寄存器时一定要查阅《STM32F4xx 参考手册》中对应外设的寄存器说明。在 GPIO 外设中，通过设置“模式寄存器 GPIOx_MODER”可配置 GPIO 的输入/输出/复用/模拟模式，“输出类型寄存器 GPIOx_OTYPER”配置推挽/开漏模式，配置“输出速度寄存器 GPIOx_OSPEEDR”可选 2/25/50/100MHz 输出速度，“上/下拉寄存器 GPIOx_PUPDR”可配置上拉/下拉/浮空模式，各寄存器的具体参数值见表 8-1。

表 8-1 GPIO 寄存器的参数配置

模式寄存器的 MODER 位[0:1]	输出类型寄存器的 OTYPER 位	输出速度寄存器的 OSPEEDR	上/下拉寄存器的 PUPDR 位[0:1]
01 -输出模式	0 -推挽模式 1 -开漏模式	00 -速度 2MHz 01 -速度 25MHz 10 -速度 50MHz 11 -速度 100MHz	00 -无上拉无下拉 01 -上拉 10 -下拉 11 -保留
10 -复用模式			
00 -输入模式	不可用	不可用	
11 -模拟功能	不可用	不可用	00 -无上拉无下拉 01 -保留 10 -保留 11 -保留

8.3 实验：使用寄存器点亮 LED 灯

本小节中，我们以实例讲解如何通过控制寄存器来点亮 LED 灯。此处侧重于讲解原理，请您直接用 KEIL5 软件打开我们提供的实验例程配合阅读，先了解原理，学习完本小节后再尝试自己建立一个同样的工程。本节配套例程名称为“使用寄存器点亮 LED 灯”，在工程目录下找到后缀为“.uvprojx”的文件，用 KEIL5 打开即可。

自己尝试新建工程时，请对照查阅《新建工程—寄存器版》章节。若没有安装 KEIL5 软件，请参考《如何安装 KEIL5》章节。

打开该工程，具体见图 8-5，可看到一共有三个文件，分别 startup_stm32f40xx.s、stm32f4xx.h 以及 main.c，接下来我们讲会对这三个工程文件进行讲解。

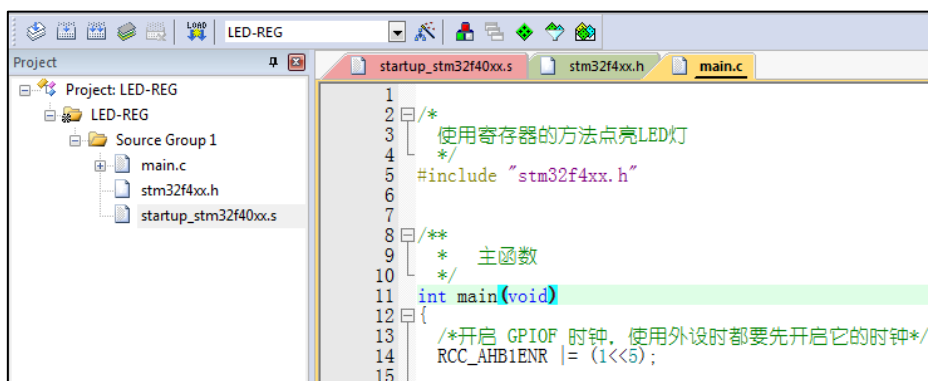


图 8-5 工程文件结构

8.3.1 硬件连接

在本教程中 STM32 芯片的 PF6、PF7 和 PF8 引脚分别与一个 RGB 灯的 R 灯（红灯）、G 灯（蓝灯）和 B 灯（绿灯）连接，具体见图 8-6。RGB 灯里面的三个小灯都可以单独控制，如果有两个或者三个灯同时亮的话就会混合成其它的颜色。

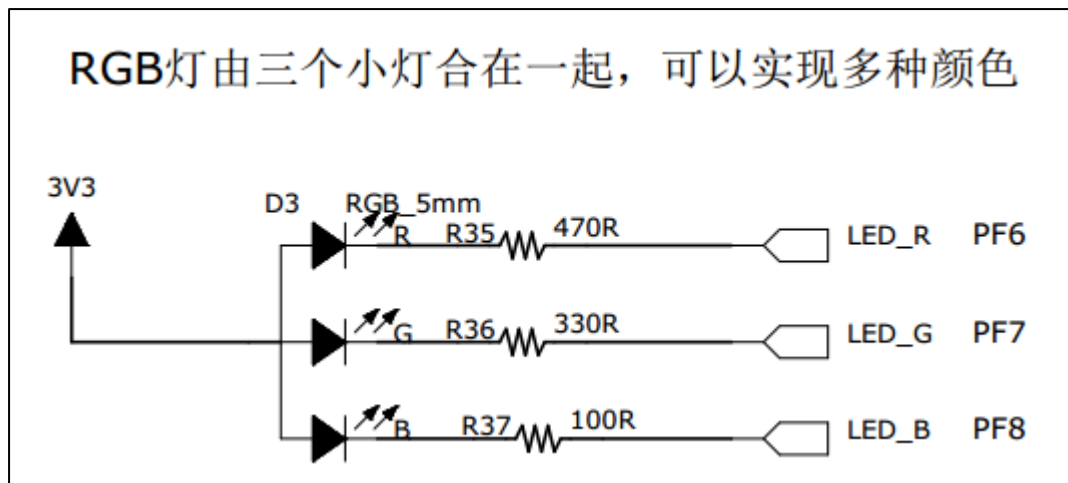


图 8-6 RGB 灯电路连接图

图 8-6 中从 3 个 LED 灯的阳极引出连接到 3.3V 电源，阴极各经过 1 个限流电阻电阻引入至 STM32 的 3 个 GPIO 引脚 PF6、PF7 和 PF8 中，所以我们只要控制这三个引脚输出高低电平，即可控制其所连接 LED 灯的亮灭。如果您的实验板 STM32 连接到 LED 灯的引脚或极性不一样，只需要修改程序到对应的 GPIO 引脚即可，工作原理都是一样的。

我们的目标是把 GPIO 的引脚设置成推挽输出模式并且默认下拉，输出低电平，这样就能让 LED 灯亮起来了。

8.3.2 启动文件

名为“startup_stm32f40xx.s”的文件，它里边使用汇编语言写好了基本程序，当 STM32 芯片上电启动的时候，首先会执行这里的汇编程序，从而建立起 C 语言的运行环境，所以我们把这个文件称为启动文件。该文件使用的汇编指令是 Cortex-M4 内核支持的指令，可从《Cortex-M4 Technical Reference Manual》查到，也可参考《Cortex-M3 权威指南中文》，M3 跟 M4 大部分汇编指令相同。

startup_stm32f40xx.s 文件是由官方提供的，一般有需要也是在官方的基础上修改，不会自己完全重写。该文件可以从 KEIL5 安装目录找到，也可以从 ST 库里面找到，找到该文件后把启动文件添加到工程里面即可。不同型号的芯片以及不同编译环境下使用的汇编文件是不一样的，但功能相同。

对于启动文件这部分我们主要总结它的功能，不详解讲解里面的代码，其功能如下：

- ☐ 初始化堆栈指针 SP;
- ☐ 初始化程序计数器指针 PC;
- ☐ 设置堆、栈的大小;
- ☐ 设置中断向量表的入口地址;
- ☐ 配置外部 SRAM 作为数据存储器（这个由用户配置，一般的开发板可没有外部 SRAM）;
- ☐ 调用 SystemIni() 函数配置 STM32 的系统时钟。
- ☐ 设置 C 库的分支入口“__main”（最终用来调用 main 函数）;

先去除繁枝细节，挑重点的讲，主要理解最后两点，在启动文件中有一段单片机复位后立即执行的程序，代码见代码清单 8-1。在实际工程中阅读时，可使用编辑器的搜索 (Ctrl+F) 功能查找这段代码在文件中的位置。

代码清单 8-1 复位后执行的程序

```
1 ;Reset handler
2 Reset_Handler    PROC
3 EXPORT Reset_Handler    [WEAK]
4     IMPORT SystemInit
5     IMPORT __main
6
7     LDR    R0, =SystemInit
8     BLX    R0
9     LDR    R0, =__main
10    BX     R0
11    ENDP
```

开头的是程序注释，在汇编里面注释用的是“;”，相当于 C 语言的“//”注释符

第二行是定义了一个子程序：Reset_Handler。PROC 是子程序定义伪指令。这里就相当于 C 语言里定义了一个函数，函数名为 Reset_Handler。

第三行 EXPORT 表示 Reset_Handler 这个子程序可供其他模块调用。相当于 C 语言的函数声明。关键字[WEAK]表示弱定义，如果编译器发现在别处定义了同名的函数，则在链接时用别处的地址进行链接，如果其它地方没有定义，编译器也不报错，以此处地址进行链接，这就正好体现了弱定义的弱，真的是弱爆了。

第四行和第五行 IMPORT 说明 SystemInit 和 __main 这两个标号在其他文件，在链接的时候需要到其他文件去寻找。相当于 C 语言中，从其它文件引入函数声明。以便下面对外部函数进行调用。

SystemInit 需要由我们自己实现，即我们要编写一个具有该名称的函数，用来初始化 STM32 芯片的时钟，一般包括初始化 AHB、APB 等各总线的时钟，需要经过一系列的配置 STM32 才能达到稳定运行的状态。使用固件库编程的时候，SystemInit 这个函数在固件库文件 system_stm32f4xx.c 中实现。

__main 其实不是我们定义的(不要与 C 语言中的 main 函数混淆)，当编译器编译时，只要遇到这个标号就会定义这个函数，该函数的主要功能是：负责初始化栈、堆，配置系统环境，准备好 C 语言并在最后跳转到用户自定义的 main 函数，从此来到 C 的世界。

第 7 行把 SystemInit 的地址加载到寄存器 R0。

第 8 行程序跳转到 R0 中的地址执行程序，即执行 SystemInit 函数的内容。

第 9 行把 __main 的地址加载到寄存器 R0。

第 10 行程序跳转到 R0 中的地址执行程序，即执行 __main 函数，执行完毕之后就去到我们熟知的 C 世界，进入 main 函数。

第 11 行表示子程序的结束。

总之，看完这段代码后，了解到如下内容即可：我们需要在外部定义一个 SystemInit 函数设置 STM32 的时钟；STM32 上电后，会执行 SystemInit 函数，最后执行我们 C 语言中的 main 函数。

8.3.3 stm32f4xx.h 文件

看完启动文件，那我们立即写 SystemInit 和 main 函数吧？别着急，定义好了 SystemInit 函数和 main 我们又能写什么内容？连接 LED 灯的 GPIO 引脚，是要通过读写寄存器来控制的，就这样空着手，如何控制寄存器。在上一章，我们知道寄存器就是特殊的内存空间，可以通过指针操作访问寄存器。所以此处我们根据 STM32 的存储器映射先定义好各个寄存器的地址，把这些地址定义都统一写在 stm32f4xx.h 文件中，见代码清单 8-2。

代码清单 8-2 外设地址定义

```
1 /*片上外设基地址 */
2 #define PERIPH_BASE          ((unsigned int)0x40000000)
3 /*总线基地址 */
4 #define AHB1PERIPH_BASE      (PERIPH_BASE + 0x00020000)
5 /*GPIO 外设基地址*/
6 #define GPIOF_BASE           (AHB1PERIPH_BASE + 0x1400)
7
8 /* GPIOH 寄存器地址,强制转换成指针 */
9 #define GPIOH_MODER          *(unsigned int*)(GPIOH_BASE+0x00)
10 #define GPIOH_OTYPER         *(unsigned int*)(GPIOH_BASE+0x04)
11 #define GPIOH_OSPEEDR        *(unsigned int*)(GPIOH_BASE+0x08)
12 #define GPIOH_PUPDR          *(unsigned int*)(GPIOH_BASE+0x0C)
13 #define GPIOH_IDR            *(unsigned int*)(GPIOH_BASE+0x10)
14 #define GPIOH_ODR            *(unsigned int*)(GPIOH_BASE+0x14)
15 #define GPIOH_BSRR           *(unsigned int*)(GPIOH_BASE+0x18)
16 #define GPIOH_LCKR           *(unsigned int*)(GPIOH_BASE+0x1C)
17 #define GPIOH_AFR1           *(unsigned int*)(GPIOH_BASE+0x20)
18 #define GPIOH_AFR2           *(unsigned int*)(GPIOH_BASE+0x24)
19
20 /*RCC 外设基地址*/
21 #define RCC_BASE              (AHB1PERIPH_BASE + 0x3800)
22 /*RCC 的 AHB1 时钟使能寄存器地址,强制转换成指针*/
23 #define RCC_AHB1ENR           *(unsigned int*)(RCC_BASE+0x30)
```

GPIO 外设的地址跟上一章讲解的相同，不过此处把寄存器的地址值都直接强制转换成了指针，方便使用。代码的最后两段是 RCC 外设寄存器的地址定义，RCC 外设是用来设置时钟的，以后我们会详细分析，本实验中只要了解到使用 GPIO 外设必须开启它的时钟即可。

8.3.4 main 文件

现在就可以开始编写程序了，在 main 文件中先编写一个 main 函数，里面什么都没有，暂时为空。

```
1 int main (void)
2 {
3 }
```

此时直接编译的话，会出现如下错误：

“Error: L6218E: Undefined symbol SystemInit (referred from startup_stm32f40xx.o)”

错误提示 SystemInit 没有定义。从分析启动文件时我们知道，Reset_Handler 调用了该函数用来初始化 STM32 系统时钟，为了简单起见，我们在 main 文件里面定义一个

SystemInit 空函数，什么也不做，为的是骗过编译器，把这个错误去掉。关于配置系统时钟我们在后面再写。当我们不配置系统时钟时，STM32 芯片会自动将内部的高速时钟 HIS（16M）作为系统时钟，程序还是能跑的，只是跑的比较慢（通常情况下，我们会把系统时钟设置为 168M，远远大于 HIS 的 16M）。我们在 main 中添加 SystemInit 函数：

```
1 // 函数为空，目的是为了骗过编译器不报错
2 void SystemInit(void)
3 {
4 }
```

这时再编译就没有错了，完美解决。

接下来在 main 函数中添加代码，对寄存器进行控制，有关 GPIO 寄存器的详细描述请参考《STM32F4xx 中文参考手册》“通用 I/O(GPIO)”章节的寄存器描述部分。

1. GPIO 模式

首先我们把连接到 RGB 红灯的 PF6 引脚配置成输出模式，即配置 GPIO 的 MODER 寄存器，具体见图 8-7。MODER 中包含 0-15 号引脚，每个引脚占用 2 个寄存器位。这两个寄存器位设置成“01”时即为 GPIO 的输出模式，具体见代码清单 8-3。

代码清单 8-3 配置输出模式

```
1 /*GPIOF MODER6 清空*/
2 GPIOF_MODER &= ~(0x03<<(2*6));
3 /*PF6 MODER6 = 01b 输出模式*/
4 GPIOF_MODER |= (1<<(2*6));
```

GPIO 端口模式寄存器 (GPIOx_MODER) (x = A..I)

GPIO port mode register

偏移地址: 0x00

复位值:

- 0xA800 0000 (端口 A)
- 0x0000 0280 (端口 B)
- 0x0000 0000 (其它端口)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

位 2y:2y+1 MODERy[1:0]: 端口 x 配置位 (Port x configuration bits) (y = 0..15)

这些位通过软件写入，用于配置 I/O 方向模式。

- 00: 输入 (复位状态)
- 01: 通用输出模式
- 10: 复用功能模式
- 11: 模拟模式

图 8-7 MODER 寄存器说明(摘自《STM32F4xx 参考手册》)

在代码中，我们先把 GPIOH MODER 寄存器的 MODER6 对应位清 0，然后再向它赋值“01”，从而使 GPIOF6 引脚设置成输出模式。

代码中使用了“&=~”、“|=”这种位操作方法是为了避免影响到寄存器中的其它位，因为寄存器不能按位读写，假如我们直接给 MODER 寄存器赋值：

```
1 GPIOF_MODER = 0x0004 0000;
```

这时 MODER6 的两个位被设置成“01”输出模式，但其它 GPIO 引脚就有意见了，因为其它引脚的 MODER 位都已被设置成 00 的输入模式。所以为了不影响寄存器的其它位，必须使用“&=~”（清 0）、“|= ”（置位）这种位操作方法来实现对寄存器的写操作。

2. 输出类型

GPIO 输出有推挽和开漏两种类型，我们了解到开漏类型不能直接输出高电平，要输出高电平还要在芯片外部接上拉电阻，不符合我们的硬件设计，所以我们直接使用推挽模式。配置 OTYPER 寄存中的 OTYPER6 寄存器位，具体见图 8-8。该位设置为 0 时 PF6 引脚即为推挽模式，具体见代码清单 8-4。

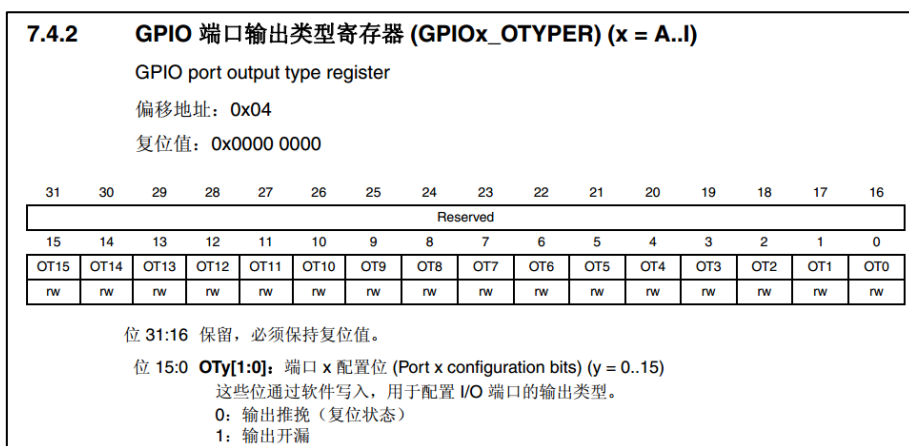


图 8-8 OTYPER 寄存器说明(摘自《STM32F4xx 参考手册》)

代码清单 8-4 设置为推挽模式

```
1 /*GPIOF_OTYPER6 清空*/  
2 GPIOH_OTYPER &= ~(1<<1*6);  
3 /*PF6_OTYPER6 = 0b 推挽模式*/  
4 GPIOH_OTYPER |= (0<<1*6);
```

3. 输出速度

GPIO 引脚的输出速度是引脚支持高低电平切换的最高频率，本实验可以随便设置。此处我们配置 OSPEEDR 寄存器中的寄存器位 OSPEEDR6 即可控制 PF6 的输出速度，寄存器描述见图 8-9，具体代码见代码清单 8-5。

7.4.3 GPIO 端口输出速度寄存器 (GPIOx_OSPEEDR) (x = A..I)

GPIO port output speed register

偏移地址: 0x08

复位值:

- 0x0000 00C0 (端口 B)
- 0x0000 0000 (其它端口)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OSPEEDR15[1:0]		OSPEEDR14[1:0]		OSPEEDR13[1:0]		OSPEEDR12[1:0]		OSPEEDR11[1:0]		OSPEEDR10[1:0]		OSPEEDR9[1:0]		OSPEEDR8[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OSPEEDR7[1:0]		OSPEEDR6[1:0]		OSPEEDR5[1:0]		OSPEEDR4[1:0]		OSPEEDR3[1:0]		OSPEEDR2[1:0]		OSPEEDR1[1:0]		OSPEEDR0[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

位 2y:2y+1 OSPEEDRy[1:0]: 端口 x 配置位 (Port x configuration bits) (y = 0..15)

这些位通过软件写入, 用于配置 I/O 输出速度。

00: 2 MHz (低速)

01: 25 MHz (中速)

10: 50 MHz (快速)

11: 30 pF 时为 100 MHz (高速) (15 pF 时为 80 MHz 输出 (最大速度))

图 8-9 OSPEEDR 寄存器说明(摘自《STM32F4xx 参考手册》)

代码清单 8-5 设置输出速度为 2MHz

```
1 /*GPIOF_OSPEEDR6 清空*/
2 GPIOH_OSPEEDR &= ~(0x03<<2*6);
3 /*PF6_OSPEEDR6 = 0b 速率 2MHz*/
4 GPIOH_OSPEEDR |= (0<<2*6);
```

4. 上/下拉模式

当 GPIO 引脚用于输入时, 引脚的上/下拉模式可以控制引脚的默认状态。但现在我们的 GPIO 引脚用于输出, 引脚受 ODR 寄存器 (数据输出寄存器) 影响, ODR 寄存器对应引脚位初始初始化后默认值为 0, 引脚输出低电平, 所以这时我们配置上/下拉模式都不会影响引脚电平状态。但因此处上拉能小幅提高电流输出能力, 我们配置它为上拉模式, 即配置 PUPDR 寄存器的 PUPDR6 位, 寄存器描述具体见图 8-10, 设置该位为二进制值“01”, 具体代码见代码清单 8-6。

7.4.4 GPIO 端口上拉/下拉寄存器 (GPIOx_PUPDR) (x = A..I)

GPIO port pull-up/pull-down register

偏移地址: 0x0C

复位值:

- 0x6400 0000 (端口 A)
- 0x0000 0100 (端口 B)
- 0x0000 0000 (其它端口)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

位 2y:2y+1 PUPDRy[1:0]: 端口 x 配置位 (Port x configuration bits) (y = 0..15)

这些位通过软件写入, 用于配置 I/O 上拉或下拉。

00: 无上拉或下拉

01: 上拉

10: 下拉

11: 保留

图 8-10 PUPDR 寄存器说明(摘自《STM32F4xx 参考手册》)

代码清单 8-6 设置为下拉模式

```
1 /*GPIOF_PUPDR6 清空*/
```

```
2 GPIOF_PUPDR &= ~(0x03<<2*6);  
3 /*PF6 PUPDR6 = 01b 上拉模式*/  
4 GPIOF_PUPDR |= (1<<2*6);
```

5. 控制引脚输出电平

在输出模式时，对 BSRR 寄存器和 ODR 寄存器写入参数即可控制引脚的电平状态。简单起见，此处我们使用 BSRR 寄存器控制，对相应的 BR6 位设置为 1 时 PF6 即为低电平，点亮 LED 灯，对它的 BS6 位设置为 1 时 PF6 即为高电平，关闭 LED 灯。寄存器 BSRR 的具体描述见图 8-11，具体代码见代码清单 8-7。

7.4.7 GPIO 端口置位/复位寄存器 (GPIOx_BSRR) (x = A..I)															
GPIO port bit set/reset register															
偏移地址: 0x18															
复位值: 0x0000 0000															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

位 31:16 **BRy**: 端口 x 复位位 y (Port x reset bit y) (y = 0..15)
这些位为只写形式，只能在字、半字或字节模式下访问。读取这些位可返回 0x0000。
0: 不会对相应的 ODRx 位执行任何操作
1: 对相应的 ODRx 位进行复位
注意: 如果同时对 BSx 和 BRx 置位，则 BSx 的优先级更高。

位 15:0 **BSy**: 端口 x 置位位 y (Port x set bit y) (y = 0..15)
这些位为只写形式，只能在字、半字或字节模式下访问。读取这些位可返回 0x0000。
0: 不会对相应的 ODRx 位执行任何操作
1: 对相应的 ODRx 位进行置位

图 8-11 BSRR 寄存器说明(摘自《STM32F4xx 参考手册》)

代码清单 8-7 控制引脚输出电平

```
1 /*PF6 BSRR 寄存器的 BR6 置 1，使引脚输出低电平*/  
2 GPIOF_BSRR |= (1<<16<<6);  
3  
4 /*PF6 BSRR 寄存器的 BS6 置 1，使引脚输出高电平*/  
5 GPIOF_BSRR |= (1<<6);
```

6. 开启外设时钟

设置完 GPIO 的引脚，控制电平输出，以为现在总算可以点亮 LED 了吧，其实还差最后一步。因为 STM32 外设很多，为了降低功耗，每个外设都对应着一个时钟，在芯片刚上电的时候这些时钟都是被关闭的，如果想要外设工作，必须把相应的时钟打开。

STM32 的所有外设的时钟由一个专门的外设来管理，叫 RCC (reset and clock control)，RCC 在《STM32F4xx 中文参考手册》的第六章有详细的讲解。

所有的 GPIO 都挂载到 AHB1 总线上，所以它们的时钟由 AHB1 外设时钟使能寄存器 (RCC_AHB1ENR) 来控制，有关该寄存器的详细描述请参考《STM32F4xx 中文参考手册》的 RCC 章节的寄存器描述部分。其中 GPIOF 端口的时钟由该寄存器的位 5 写 1 使能来开启。有关 STM32 的时钟系统我们在往后的 RCC 章节会详细的讲解，此处我们只需知道在访问 GPIO 这个外设的寄存器之前，要先开启它的时钟。具体代码见代码清单 8-8。

代码清单 8-8 开启端口时钟


```
1 /*开启 GPIOF 时钟，使用外设时都要先开启它的时钟*/  
2 RCC_AHB1ENR |= (1<<5);
```

7. 水到渠成

开启时钟，配置引脚模式，控制电平，经过这三步，我们总算可以控制一个 LED 了。
现在我们完整组织下用 STM32 控制一个 LED 的代码，见代码清单 8-9。

代码清单 8-9 main 文件中控制 LED 灯的代码

```
1 /*  
2 使用寄存器的方法点亮 LED 灯  
3 */  
4 #include "stm32f4xx.h"  
5  
6  
7 /**  
8  * 主函数  
9  */  
10 int main(void)  
11 {  
12     /*开启 GPIOF 时钟，使用外设时都要先开启它的时钟*/  
13     RCC_AHB1ENR |= (1<<5);  
14  
15     /* LED 端口初始化 */  
16  
17     /*GPIOF MODER6 清空*/  
18     GPIOF_MODER &= ~(0x03<<(2*6));  
19     /*PF6 MODER6 = 01b 输出模式*/  
20     GPIOF_MODER |= (1<<(2*6));  
21  
22     /*GPIOF OTYPER6 清空*/  
23     GPIOF_OTYPER &= ~(1<<(1*6));  
24     /*PF6 OTYPER6 = 0b 推挽模式*/  
25     GPIOF_OTYPER |= (0<<(1*6));  
26  
27     /*GPIOF OSPEEDR6 清空*/  
28     GPIOF_OSPEEDR &= ~(0x03<<(2*6));  
29     /*PF6 OSPEEDR6 = 0b 速率 2MHz*/  
30     GPIOF_OSPEEDR |= (0<<(2*6));  
31  
32     /*GPIOF PUPDR6 清空*/  
33     GPIOF_PUPDR &= ~(0x03<<(2*6));  
34     /*PF6 PUPDR6 = 01b 上拉模式*/  
35     GPIOF_PUPDR |= (1<<(2*6));  
36  
37     /*PF6 BSRR 寄存器的 BR6 置 1，使引脚输出低电平*/  
38     GPIOF_BSRR |= (1<<(16<<6));  
39  
40     /*PF6 BSRR 寄存器的 BS6 置 1，使引脚输出高电平*/  
41     //GPIOF_BSRR |= (1<<6);  
42  
43     while (1);  
44  
45 }  
46  
47 // 函数为空，目的是为了骗过编译器不报错  
48 void SystemInit(void)  
49 {  
50 }  
51
```

在本章节中，要求完全理解 `stm32f4xx.h` 文件及 `main` 文件的内容(RCC 相关的除外)。

8.3.5 下载验证

把编译好的程序下载到开发板并复位，可看到板子上的 LED 红灯被点亮。